



NLP Foundation for Generative AI



- [~2 min]
- Welcome to ICT 304. The subtitle matters more than the code: NLP Foundation for Generative AI. Not "Introduction to Generative AI." Foundation.
- Everyone in this room has used a large language model. Most of you could build something useful with one by Friday without attending a single lecture. So why are we here? Because the gap between people who use these systems and people who can diagnose, evaluate and improve them is where every job worth having sits. That gap is made of the material in this module.
- A quick word about me before we start, and then we'll get straight into it.

Learning outcome

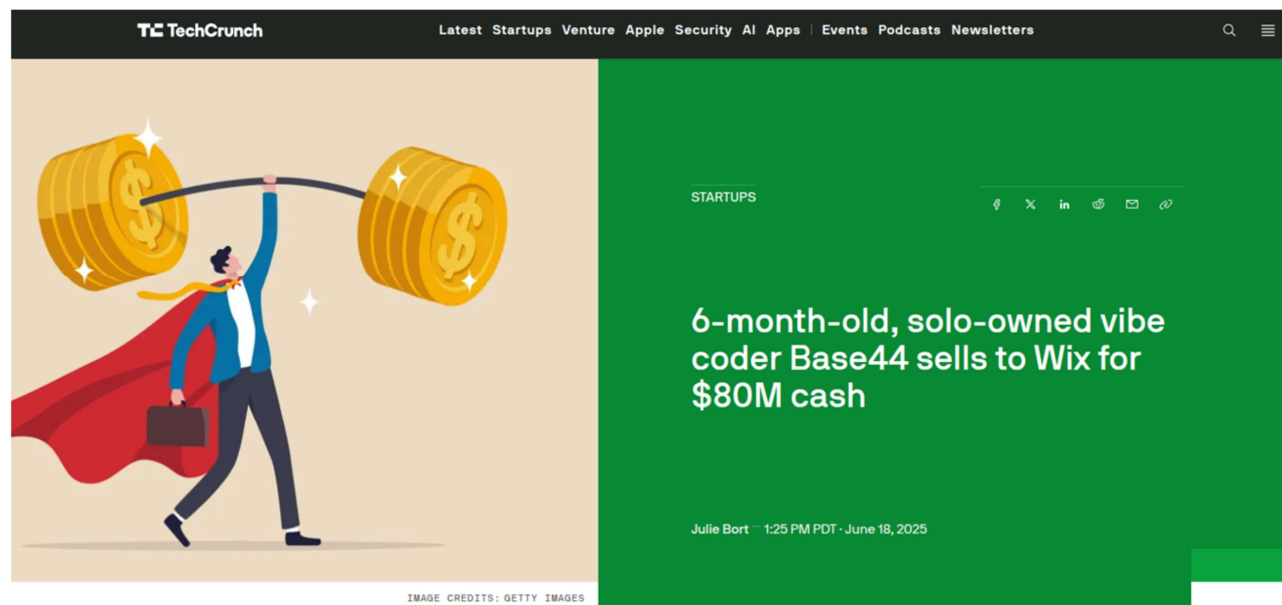
- ❑ The foundations of modern deep learning methods for NLP
 - Start with essentials: word embeddings, feed-forward networks, recurrent networks, attention
 - Then cover 2014 advancements: encoder–decoder architectures and Transformers
- ❑ A big-picture view of human language and the challenges of modeling understanding and generation on computers
- ❑ The knowledge and skills to build PyTorch systems for major NLP tasks:
 - Word meaning, dependency parsing, document classification, machine translation, and question answering

- [~7 min]
- Read the first bullet as a build order, not a menu. Word embeddings, feed-forward networks, recurrent networks, attention, then encoder-decoder and Transformers. Every step on that list exists because the previous one broke on something. If you understand why attention had to be invented, the Transformer is almost boring. If you skip straight to Transformers, you memorise a diagram.
- Encoder-decoder is 2014, and the Transformer paper is 2017. What is recent is the scale, not the architecture. Hold on to that distinction — it will come up in your assessments.
- Second bullet — the big-picture view of human language. Students undervalue this one and later wish they hadn't. Language is ambiguous, compositional, and infinite in a way images are not. A model that says something fluent and wrong is a specific engineering failure, and you need vocabulary for it. We spend a whole section on that today.
- Third bullet — you will write PyTorch. Not read it, write it. Word meaning, dependency parsing, document classification, machine translation, question answering.

h | e | f | u | t | u | r | e | s | S | o | l | o

Vibe Coding + Agentic AI

- [~1 min]
- Before any technical content, we need to agree on how you are going to work in this module. This section is about the tools you will use every day and what I expect from you when you use them. It is not a tangent.



- [~4 min]
- June 2025. Base44 — six months old, one owner, no co-founder — sells to Wix for eighty million dollars in cash.
- Let me be careful about the lesson here. The lesson is not "ship fast with AI and get rich." Outliers make bad curricula. The lesson is that the bottleneck moved. Three years ago the constraint on a product like this was implementation headcount. It is not anymore.
- So the question becomes: if implementation is cheap, what is expensive?
- Have a think about that. I will answer it in three slides.

Vibe Coding

<https://youtu.be/LCEmiRjPEtQ>

- [~4 min]
- This is Karpathy's coinage. Vibe coding means you stop reading the diffs. You describe what you want, you accept what comes back, and you keep going.
- That is fine for a weekend project. It is a career-limiting habit for anything with users, and I will explain why when we get to assessment.
- Let me play a couple of minutes of this and then we'll move on.

Vibe Coding

- ☐ Acts as an assistant
- ☐ Emulates a junior partner or tool enhancing productivity
- ☐ Reactive

- ☐ Acts as a collaborator
- ☐ Emulates a human peer working together on equal footing
- ☐ Autonomous

- [~5 min]
- Two modes here, and the real difference is about who holds the plan.
- The left column is pair programming — Copilot, Cursor, Windsurf. The tool is reactive. It completes what you started. You hold the plan and it accelerates your typing. It is a very good junior partner and a very bad architect.
- The right column is peer programming — Codex, Jules. You hand over a goal and it works autonomously. It holds a plan. That is genuinely different, and it fails differently: instead of one wrong line you get a confidently wrong hundred lines, wired together, that runs.
- Here is the working split I would suggest, and it is what I use myself: architecture decisions with the strongest model you have, in a conversation, deliberately — and execution delegated. The model that writes your code should not be the model that decides your design. Keep those two separate and you will debug half as much.

If English is the new programming language...

- ❑ Need to go upstream
 - creator of tools
 - be the problem owner
 - discover problems
- ❑ Piece everything together
 - Extreme breadth
 - Potential to do deep quickly
 - Be brave

- [~5 min]
- This answers the question I left open three slides ago. If implementation is cheap, value moves upstream. Three things.
- Be the creator of tools, not the consumer. Consumers of AI tools are a commodity. Builders are not.
- Own the problem. The scarce skill is knowing which problem is worth solving and what "solved" actually means. No model will tell you that.
- Discover problems. Nobody hands you a well-specified ticket in the real world.
- Second half of the slide — piece everything together. You now have extreme breadth available to you. You can go deep quickly in a field you have never touched. The constraint is no longer knowledge access; it is judgment and nerve. That is what "be brave" means. Practically: attempt things one level beyond what you think you can do, because the cost of attempting has collapsed.



- [~1 min]
- Now the part that affects your grade. Please pay more attention to this section than the last one.

AI First Teaching & Learning

- Programming with Gemini, OpenAI
- Focus on the enduring concepts, frameworks, schemas
- Assessment focus on consistency and progression
- Don't lose sight of *your* goal

- [~9 min]
- Bullet by bullet, because you need to know exactly what is permitted.
- Programming with Gemini and OpenAI — permitted and expected. I am not going to pretend you will not use them. Using them well is part of the skill I am assessing.
- Focus on the enduring concepts, frameworks and schemas. Do not burn a week on syntax you can look up. That does not mean don't understand your code. If you submit something you cannot explain line by line, you have failed the assessment even if it runs.
- Assessment focuses on consistency and progression. I am not only marking the final artefact. I am looking at the commit history. Twenty commits over eight weeks tells me a story. One commit at 2am the night before tells me a different story, and I will believe it.
- Don't lose sight of your goal. Concretely: when you pick your project, pick something you would still want to exist if it were not graded.
- Someone is about to ask whether they can just have the AI write it. The answer: use it however you like, I will ask you to explain any part of it, and the commit history has to show a human learning. Those three constraints together are the policy.

Machine learning I

- [~1 min]
- Gear change. The rest of today is fundamentals. Some of you have seen this material before — sit with it anyway, because I am going to set up notation we use all semester.

What is Machine learning?

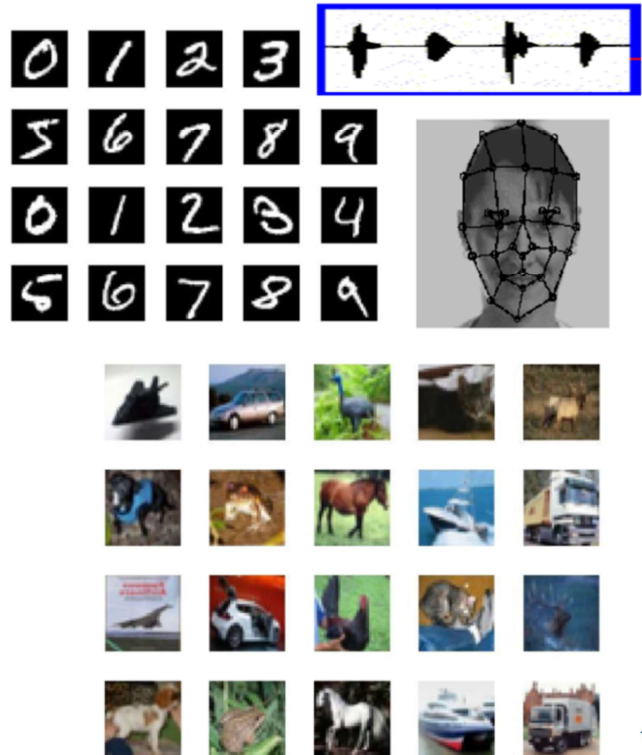
❑ Machine learning

- is a subfield of computer science
- that is concerned with building algorithms which,
- rely on a collection of examples of some phenomenon.
- ✓ These examples can come from nature, be handcrafted by humans or generated by another algorithm.

- [~4 min]
- The definition on the slide is deliberately dry. The load-bearing word is "examples."
- Classical programming: you write the rules, data goes in, answers come out.
Machine learning: data and answers go in, and the rules come out. That inversion is the entire idea.
- Note the last line — examples can come from nature, be handcrafted by humans, or be generated by another algorithm. That third one used to be a footnote. It is now a major research area: synthetic data, distillation, self-instruct. Remember that this line was on a slide in week one.

Application examples

- Speech recognition
- Face recognition
- Handwriting recognition
- Object recognition
- Housing price prediction
- Etc, etc.



- [~3 min]
- I am not going to read this list. I want you to notice one thing about it.
- Speech, faces, handwriting, objects — those four output a category. House price outputs a number. That is a different kind of problem, and the distinction has a name. Hold the thought for one slide.

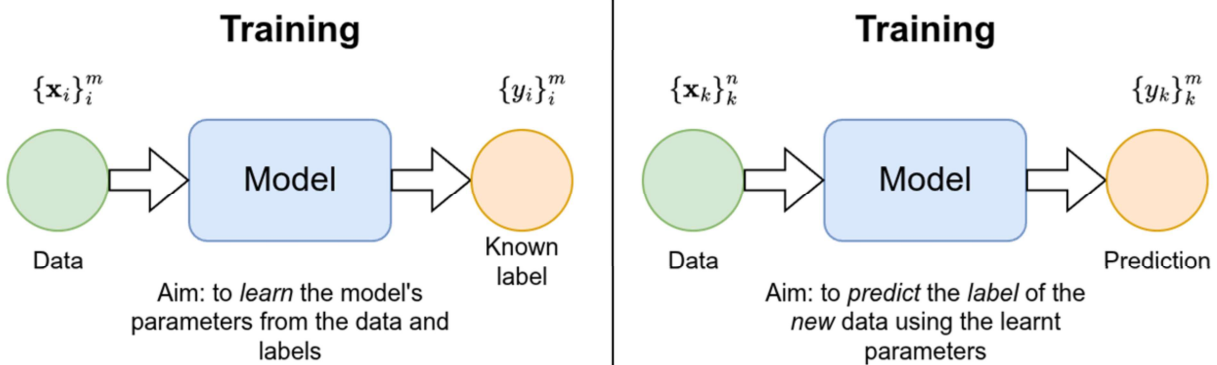
Types of Learning

- Supervised Learning
- Unsupervised Learning
- Semi-Supervised Learning
- Reinforcement Learning

	Supervised Learning	Unsupervised Learning
Discrete	Classification	Clustering
Continuous	Regression	Dimensionality Reduction

- [~6 min]
- Four types on the left. The two-by-two grid on the right is the better artefact, and it is the thing to memorise.
- Columns: do you have labels? Supervised, yes. Unsupervised, no.
- Rows: is the output discrete or continuous?
- Discrete plus labels is classification. Continuous plus labels is regression. Discrete without labels is clustering. Continuous without labels is dimensionality reduction. That is your house price question from the last slide, answered.
- Two of the types on the left are not in the grid. Semi-supervised — a few labels, mostly not. Extremely common in practice, because labelling is expensive. Reinforcement learning — no labels at all, just a reward signal from an environment. RL does not fit the grid because it is a different problem shape: sequential decisions rather than one-shot predictions. It is also how RLHF works, so we will come back to it.
- One more thing, and please write it in, because it is not on the slide. Where does language modelling sit? A language model is trained supervised — predict the next token, and the label is the next token. But nobody labelled anything; the label came free from the raw text. That is self-supervised learning, and it is the reason we can train on the internet.

How Supervised Learning Works



- [~4 min]
- Left panel: training. Data plus known labels go into the model, and we learn parameters.
- The right panel is labelled "Training" as well — that is a typo on the slide. It should say inference, or testing. Data goes in, no labels, a prediction comes out, using the parameters we already learned.
- The distinction is not pedantic. The single most common mistake in applied machine learning is letting information from the right-hand box leak into the left-hand box. We will name that properly when we get to data splits.

Supervised Learning

In supervised learning, the dataset is the collection of labeled examples

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^m$$

$$\mathbf{x}_i = [x_1, x_2, \dots, x_d]^T, i = 1, \dots, m$$

- Each element $\mathbf{x}_i$ among m is called a feature vector.
 - A feature vector is a vector in which each dimension contains a value that describes the example somehow.
- The label y_i can be either an element belonging to a finite set of classes $\{1, 2, \dots, C\}$, or a real number.

- [~6 min]
- Slow down here. This is the notation for the semester.
- The dataset is a collection of m labelled examples, each one a pair: x_i and y_i .
- Each x_i is a feature vector — d numbers, each describing the example somehow. d is the dimensionality.
- y_i is the label: either an element of a finite set of classes, or a real number. Classification or regression — the same split from two slides ago.
- Two numbers to keep straight all semester. m is how many examples you have. d is how many features each one has. When somebody says "more data," ask which one they are increasing.
- And here is the question this whole module answers: text is not numbers. A sentence is not a vector. So how do we get from an email to an x ? That is the second half of today, and honestly, that is the module.

Supervised Learning

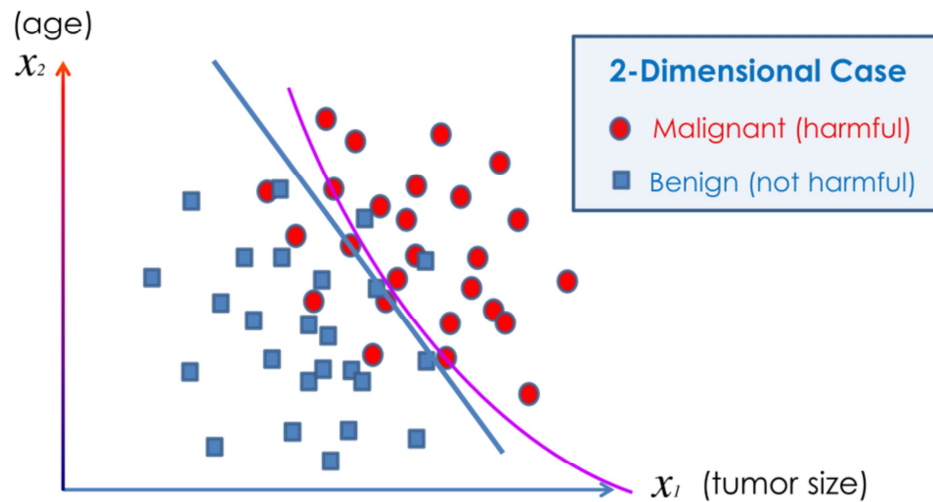
- For instance, if your examples are email messages and your problem is spam detection, then you have two classes {spam, not-spam}.
- **Classification:** predict discrete valued output (e.g., 0 or 1, spam or ham)

Free entry in 2 a wkly
comp to win FA Cup final
tkts 21st May 2005. Text
FA to 87121 to receive
entry question(std txt
rate)T&C's apply
08452810075over18's

→ Ham or Spam

- [~4 min]
- Concrete version. Examples are emails, classes are spam and not-spam. Classification — discrete output. Zero or one, spam or ham.
- Let me read the message on the slide out loud, because it is genuinely funny in 2026.
- Notice what makes it spam to you: "Free," "win," a shortcode, a premium-rate number, "T&C's apply." Those are features. You just did feature engineering in your head in about half a second. That is the thing we are going to try to automate.
- And hold on to the word "ham." It is the standard term for non-spam, it predates the field, and nobody is entirely sure why.

Classification: Breast Cancer (malignant, benign)



- [~5 min]
- Two features — tumour size on the x-axis, age on the y-axis. Two classes. Two candidate decision boundaries drawn: a straight line and a curve.
- Which one is better?
- [Take answers. Most will say the curve.]
- The curve fits the training data better. Whether it is better depends entirely on data you have not seen. That is the overfitting question, and it is the central tension in this field.
- One more thing, and it matters. This is a medical example, so the two errors are not equal. Calling a malignant tumour benign is not the same mistake as calling a benign tumour malignant. Accuracy treats them identically. Keep that discomfort — you will need it every time you report a number.

Unsupervised Learning

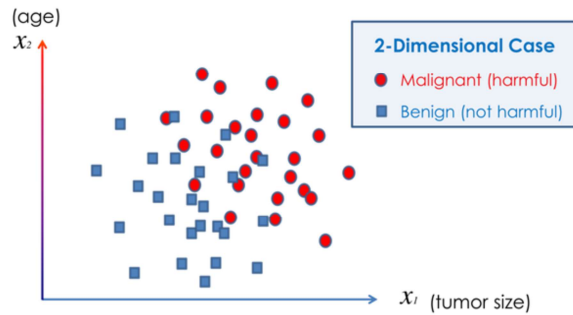
- In unsupervised learning, the dataset is a collection of unlabeled examples

$$\{\mathbf{x}_i\}_{i=1}^m$$

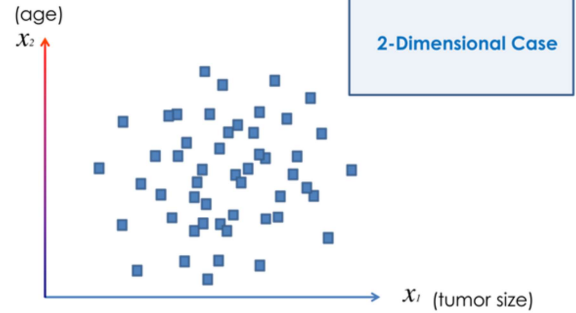
- Again, $\mathbf{x}$ is a feature vector, and the goal of an **unsupervised learning algorithm** is to create a **model** that takes a feature vector $\mathbf{x}$ as input and either **transforms it into another vector or into a value** that can be used to solve a practical problem.

- [~4 min]
- Same setup, labels removed. Just the $\mathbf{x}$'s, m of them.
- The goal: take a feature vector and either transform it into another vector, or into a value, such that the result solves some practical problem.
- That definition sounds vague because it is. Unsupervised learning is a much less well-posed problem — there is no answer key, so "did it work?" is a genuinely hard question.
- Flag for later: "transforms it into another vector" is exactly what an embedding is. Word2Vec, which we cover in week three, is unsupervised learning by this definition.

Supervised Learning

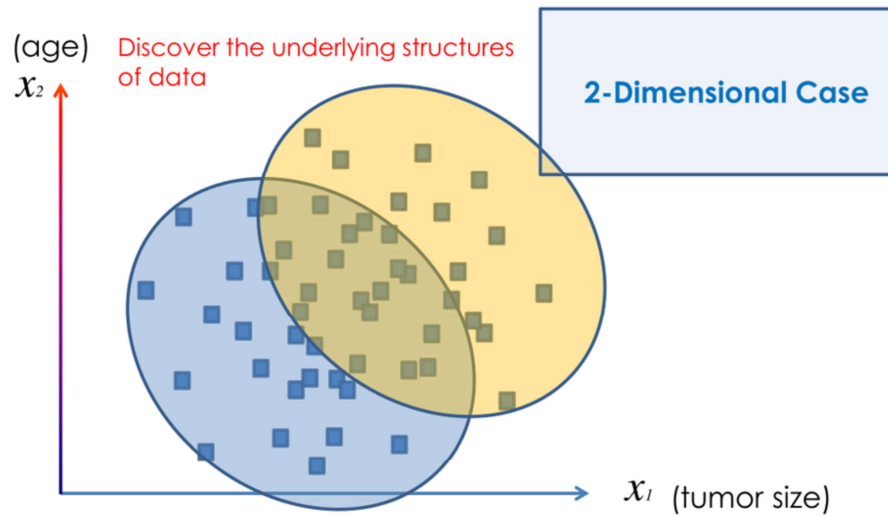


Un-Supervised Learning



- [~2 min]
- Same data, twice. On the left I have told you the colours. On the right I have not. That is the whole difference — and everything downstream follows from whether somebody paid to label your data.

Un-Supervised Learning: Clustering



- [~4 min]
- Unsupervised learning on the same points: discover the underlying structure. Two clusters found.
- Look at the overlap in the middle — the clusters intersect. Two honest observations. First, real clusters overlap, so hard assignment is often a lie, which is why soft clustering exists. Second, and more important: nothing here says these clusters are malignant and benign. The algorithm found structure. What that structure means is your interpretation, and interpretation is where people over-claim.
- When we do topic modelling later, this warning comes back word for word.



- [~1 min]
- That is the machine learning scaffolding. Now let's put language into it, and be precise about what NLP actually is.

What Is NLP? (And What It Is Not)

- NLP sits at the intersection of linguistics, computer science, and statistics
 - The goal: enable machines to process, understand, and generate human language
- Distinguishing NLP from adjacent fields:
 - Computer vision -- images, not text
 - Classical symbolic AI -- rule-based logical inference
 - Information retrieval -- finding documents, not understanding them

- [~5 min]
- NLP sits at the intersection of linguistics, computer science and statistics. The goal is to enable machines to process, understand and generate human language.
- The more useful part of this slide is the second half — what NLP is not.
- Computer vision: images, not text. Different data, and a much less ambiguous signal.
- Classical symbolic AI: rule-based logical inference. Historically overlapping with NLP, and we will see why it lost.
- Information retrieval: finding documents, not understanding them. This one is subtle and important, because search is the application people most often mistake for language understanding. A search engine that returns the right page has not understood your question. It has matched it.
- Keep that boundary in mind — it is exactly the boundary that retrieval-augmented generation tries to blur.

Core NLP Task Types

- Text classification -- predict a document-level label
 - Spam email detection, sentiment classification
- Sequence labelling -- predict a label per token
 - Part-of-speech tagging (POS), named entity recognition (NER)
- Structured prediction -- predict a structure over the whole input
 - Dependency parsing, coreference resolution
- Language generation -- produce new text
 - Machine translation, summarisation, dialogue/chatbot response
- Question answering -- extractive (find the span) or generative (produce the answer)

- [~7 min]
- Five task types. This is the taxonomy I want you to be able to reproduce from memory, because every system you meet for the rest of the course is one of these.
- Text classification — one label for a whole document. Spam, sentiment. Simplest output shape.
- Sequence labelling — one label per token. POS tagging, named entity recognition. The output is the same length as the input.
- Structured prediction — a structure over the whole input. Dependency parsing, coreference. Now the output is a graph, not a list, and evaluation gets harder.
- Language generation — produce new text. Translation, summarisation, dialogue. The output length is not determined by the input length. This is one reason encoder–decoder architectures became especially useful for tasks such as machine translation and summarisation.
- Question answering — and notice it splits in two. Extractive: find the span in a document. Generative: produce the answer. Those are very different problems with very different failure modes. An extractive system can be wrong; a generative system can be wrong and invent the evidence.
- Notice the tasks are ordered by how constrained the output is. Constrained outputs are easier to evaluate. That ordering is not an accident and it will keep coming back.

Why language is hard

Ambiguity, and the NLP Pipeline



- [~1 min]
- Now the part that makes language different from every other kind of data. This is the conceptual heart of week one.

Sources of Ambiguity

- Lexical ambiguity
 - "bank" — a financial institution, or a river bank?
- Syntactic ambiguity
 - "I saw the man with the telescope." — who has the telescope?
- Semantic ambiguity
 - "Every student read a book." — the same book, or one each?
- Coreference
 - "Mary told Sue she was late." — who was late?
- World knowledge
 - "The chicken was too hot to eat." — the chicken is food, not an animal in distress

- [~8 min]
- Five kinds of ambiguity, and I want to work them rather than read them.
- Lexical: "bank." Financial institution or river bank? One word, two meanings, and you resolved it instantly from context you were not conscious of using.
- Syntactic: "I saw the man with the telescope." Who has the telescope? The grammar genuinely permits both. This is not a badly written sentence; it is a correctly written sentence with two parse trees.
- Semantic: "Every student read a book." Same book, or one each? Both readings are grammatical and the difference is a quantifier scope — and it changes how many books exist.
- Coreference: "Mary told Sue she was late." Who was late? Nothing in the sentence resolves it. You need context, or you need to guess.
- World knowledge: "The chicken was too hot to eat." You know the chicken is food and not an animal in distress. Nothing in the syntax tells you that. You brought it.
- Here is the point. Every one of these is trivial for you and hard for a machine, and in every case the thing that resolved it was knowledge that was not in the string. That is why language is hard.

Ambiguity in Action

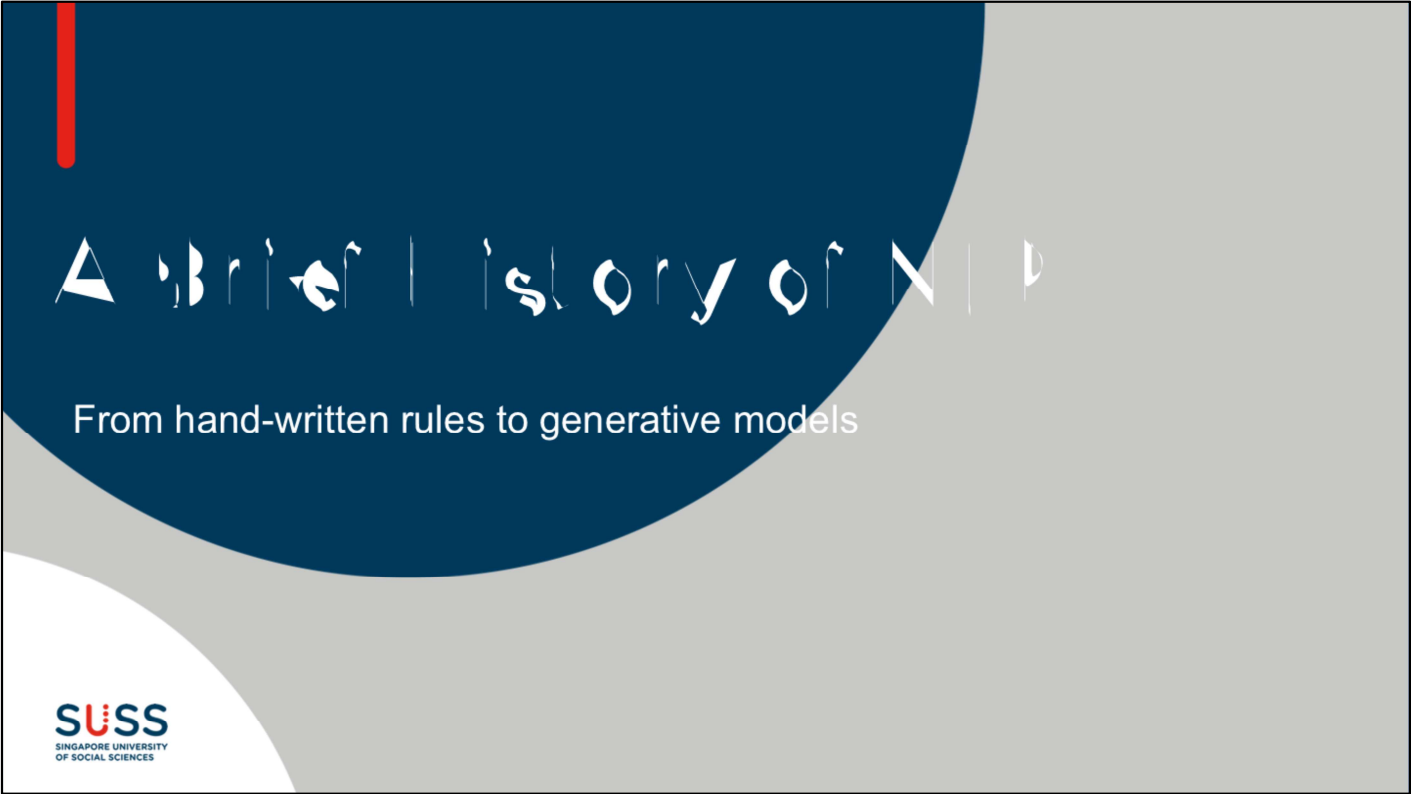
- "Time flies like an arrow; fruit flies like a banana."
 - "flies" is ambiguous — noun (fruit flies) or verb (time flies)?
 - Resolving it requires world knowledge (fruit flies are insects), not just grammar
- Pattern matching is not understanding
 - A model can learn "Paris is the capital of France" — and just as confidently assert a false claim if it appears in training data
- What would "understanding" actually require?
 - Grounding in the physical world, common-sense reasoning, distinguishing assertion from implication
 - No model today fully achieves this -- worth being honest about from Week 1

- [~7 min]
- "Time flies like an arrow; fruit flies like a banana."
- The joke works because "flies" flips from verb to noun and "like" flips from preposition to verb between the two clauses. And notice what resolves it: you know fruit flies are insects. Grammar alone does not get you there. World knowledge does.
- Second half of the slide, and this is the honest bit. Pattern matching is not understanding. A model can learn "Paris is the capital of France," and it will assert a false claim with exactly the same confidence if that claim appeared in its training data. There is no internal flag that distinguishes the two. That is worth sitting with.
- So what would understanding actually require? Grounding in the physical world. Common-sense reasoning. Distinguishing an assertion from an implication.
- No model today fully achieves this. I want to be straight with you about that in week one, because you will spend the next eleven weeks building things that are extremely useful and are still not doing the thing the marketing says they are doing. Holding both of those at once is the professional skill.



- Raw text → Tokens → Representations → Task output
 - Raw text: the original string, exactly as written
 - Tokens: text split into units a model can consume
 - Representations: tokens turned into vectors a model can compute with (Weeks 3, 9)
 - Task output: the label, span, or sequence the model produces
- Every system built in this course lives somewhere on this stack

- [~5 min]
- This is the map for the whole module. Four stages.
- Raw text — the original string, exactly as written, including the typos and the emoji and the inconsistent whitespace.
- Tokens — text split into units a model can consume. That is the second half of today.
- Representations — tokens turned into vectors a model can compute with. Weeks three and nine.
- Task output — the label, span or sequence the model produces.
- Every system we build in this course lives somewhere on this stack. When something breaks — and things will break in your TMA — the first diagnostic question is always: which stage did this go wrong at? Most people jump straight to the model. Most of the time it is stage one or stage two.



A Brief History of NLP

From hand-written rules to generative models

- [~1 min]
- Quick history. Four eras, about six minutes, and the reason I am doing it is that each era failed in a way that explains the next one.

Four Eras of NLP

- Rule-based (1950s–1990s)
 - Handcrafted grammars and pattern matching; ELIZA (1966) simulated a therapist by reflecting questions back — brittle outside its rules
- Statistical NLP (1990s–2010s)
 - Learn patterns from corpora: Hidden Markov Models, n-gram language models — data-driven but shallow
- Neural NLP (2013–2018)
 - Word embeddings (Word2Vec, GloVe), recurrent networks, seq2seq with attention — representations learned from data
- Pre-trained language models (2018–present)
 - BERT, GPT, T5: pre-train on massive corpora, then fine-tune for a task

- [~7 min]
- Rule-based, 1950s to 1990s. Handcrafted grammars and pattern matching. ELIZA in 1966 simulated a therapist by reflecting your questions back at you, and people genuinely confided in it. Brittle the moment you stepped outside its rules. Why it failed: you cannot hand-write enough rules to cover a language.
- Statistical NLP, 1990s to 2010s. Learn the patterns from corpora instead. Hidden Markov Models, n-gram language models. Data-driven, but shallow — an n-gram model has no idea what a word means, only what tends to follow it. Why it failed: no generalisation between similar words.
- Neural NLP, 2013 to 2018. Word embeddings — Word2Vec, GloVe — recurrent networks, sequence-to-sequence with attention. Now representations are learned from data, and "good" and "great" finally live near each other. Why it hit a ceiling: recurrence is sequential, so it does not parallelise, and long-range dependencies decay.
- Pre-trained language models, 2018 to now. BERT, GPT, T5. Pre-train on a massive corpus, then fine-tune for your task. The insight is that most of what a model needs to know about language is task-independent, so learn it once.
- Four eras, and each transition is a specific failure being fixed. That is the shape of the story.

Where This Course Sits

- ICT304 covers the full arc, Weeks 1–12
 - TMA (Weeks 1–6): the statistical and early neural era
 - ECA (Weeks 9–12): the pre-trained transformer era
- What does "GPT" stand for?
 - Generative Pre-trained Transformer
 - Generative → Week 7 onward, Pre-trained → Week 9, Transformer → Week 8

- [~4 min]
- Where this course sits. ICT304 covers the full arc across weeks one to twelve.
- Your TMA, weeks one to six, sits in the statistical and early neural era. Your ECA, weeks nine to twelve, sits in the pre-trained transformer era. So the two assessments are not arbitrary chunks — they are the two halves of the history you just saw.
- Quick question for the room: what does GPT stand for?
- Generative, Pre-trained, Transformer. And look at how the module is built: Generative is week seven onward, Pre-trained is week nine, Transformer is week eight. By the time you have finished week nine, you will be able to explain all three words from first principles. That is the design of the course in one acronym.

Common Pitfalls

- Common pitfalls
 - Conflating NLP with chatbots -- chatbots are one application; most NLP is invisible (spam filters, autocorrect, search ranking)
 - Treating "understanding" as binary -- ask what the model can do and how it fails, not yes/no
 - Underestimating tokenisation -- `text.split()` looks fine until it meets "don't", numbers, or URLs
- Where this shows up in your assessments
 - TMA Q1a: data audit on AG News -- the pipeline framing from today applies directly
 - ECA Part A: data audit on the CFPB complaints dataset -- same skill, different data

- [~5 min]
- Three pitfalls, and I raise them now because I will see all three in your submissions if I do not.
- Conflating NLP with chatbots. Chatbots are one application. Most NLP is invisible — spam filters, autocorrect, search ranking. You use twenty NLP systems before lunch and notice none of them.
- Treating understanding as binary. Do not ask "does the model understand." Ask what it can do and how it fails. The second question has answers; the first one does not.
- Underestimating tokenisation. `text-dot-split` looks fine right up until it meets "don't", or a number, or a URL. That is the whole next section— important to understand

Text Representation: Tokenisation and Subword Methods

- [~1 min]
- Break here if we need one. Then: stage two of the pipeline. We have been saying "word" all morning as though it is obvious what one is. It is not.

Why Whitespace Splitting Fails

- Punctuation
 - "Hello, world!" → ["Hello,", "world!"] — punctuation stays glued to words
- Contractions
 - "don't" is morphologically do + not, but whitespace keeps it as one token
- Hyphenation
 - "state-of-the-art" — one word, or four?
- Numbers and special characters
 - "COVID-19", "\$3.50", "3:45pm" all carry structure that affects meaning
- Code
 - "if x > 0:" — the colon is Python syntax, not sentence punctuation
- Multilingual text
 - Chinese, Japanese, and Thai have no whitespace between words at all
- Board exercise: "The U.S. economy grew 2.3% in Q3." -- there is no single "correct" split; the right answer depends on the task

- [~9 min]
- Six reasons whitespace splitting fails. Work them out loud.
- Punctuation. "Hello, world!" splits into "Hello," with a comma glued on and "world!" with an exclamation mark glued on. Now those are different tokens from "Hello" and "world," and your vocabulary has doubled for no reason.
- Contractions. "don't" is morphologically do plus not — two words, one of them a negation, which is about as semantically important as it gets. Whitespace keeps it as one token and the negation disappears into a string.
- Hyphenation. "state-of-the-art" — one word or four? Genuinely unclear.
- Numbers and special characters. "COVID-19", "\$3.50", "3:45pm" all carry internal structure that affects meaning.
- Code. "if x > 0:" — that colon is Python syntax, not sentence punctuation. If you are training on code, and everyone is now, this matters.
- Multilingual text. Chinese, Japanese and Thai have no whitespace between words at all. Whitespace splitting does not degrade gracefully there; it fails completely.
- Now the board exercise. "The U.S. economy grew 2.3% in Q3." Take two minutes with the person next to you and write down your split.
- [Take two or three answers, write them up, let them disagree.]
- Notice there is no single correct answer. Is "U.S." one token or two? Is "2.3%" one token or two? The right answer depends on the task. That is the honest situation, and it is why tokenisation is a design decision rather than a preprocessing step.

Character vs. Word Tokenisation

```
import tiktoken
enc =
tiktoken.get_encoding("o200k_base")
assert enc.decode(enc.encode("hello
world")) == "hello world"

enc =
tiktoken.encoding_for_model("gpt-4o")
```

- Character tokenisation
 - Solves out-of-vocabulary (OOV) completely
 - but sequences get ~5x longer, with no word-level abstraction
- Word tokenisation
 - Manageable sequence lengths — but any word unseen in training becomes UNK
- The subword insight
 - Frequent words stay whole tokens; rare or novel words split into common, reusable pieces
- Try it yourself with a real subword tokeniser (tiktoken, OpenAI's BPE):
 - <https://www.kaggle.com/code/greyisbetter/tokenization-word-subword>

- [~8 min]
- Two obvious alternatives, and both are bad.
- Character tokenisation solves out-of-vocabulary completely — every string is representable. But sequences get roughly five times longer, and you have thrown away all word-level abstraction. The model now has to learn spelling from scratch.
- Word tokenisation gives you manageable sequence lengths, but any word unseen in training becomes UNK. And notice the second consequence: the model cannot generate a word that is not in its vocabulary either.
- The subword insight is the compromise, and it is genuinely elegant. Frequent words stay whole tokens. Rare or novel words split into common, reusable pieces. Fixed vocabulary size, and zero out-of-vocabulary, because in the worst case you fall back to characters.
- Now let's actually run it. The code on the slide is tiktoken — that is OpenAI's BPE implementation, and o200k_base is the encoding GPT-4o uses.
- [Run the demo. Tokenise, in this order: "hello world" for the baseline; "unbelievable" to show subword splitting; "Singapore" versus a less common place name to show frequency driving granularity; a line of Chinese or Malay to show the token count per character is far worse than English — which is a real fairness and cost issue, non-English users pay more per API call; and a long number to show numbers chunk unpredictably.]
- And the payoff. Ask a model how many R's are in "strawberry." Current models handle it, but the reason it ever failed is on this screen: the model never saw the letters. It saw two or three subword tokens. Asking it to count characters is like asking you to count the brushstrokes in a word you are reading.

- That is not a bug in reasoning. It is a consequence of the representation — and that is the level I want you thinking at for the rest of this module. Not "the AI is dumb." Which layer of the pipeline made this inevitable?
- The Kaggle notebook at the bottom is your homework.

Byte-Pair Algorithm

- Start: vocabulary = every individual character + an end-of-word symbol
- 1. Count the frequency of every adjacent pair of symbols in the corpus
- 2. Find the single most frequent pair
- 3. Merge that pair into a new token; add it to the vocabulary
- 4. Repeat steps 1–3 until the vocabulary reaches the target size

- [~5 min]
- Byte-Pair Encoding. Originally a compression algorithm from 1994, repurposed for NLP in 2015. Four steps and it is genuinely this simple.
- Start with a vocabulary of every individual character, plus an end-of-word symbol.
- One: count the frequency of every adjacent pair of symbols in the corpus.
- Two: find the single most frequent pair.
- Three: merge that pair into a new token and add it to the vocabulary.
- Four: repeat until the vocabulary reaches the target size you chose.
- Two things to notice. The target vocabulary size is a hyperparameter you pick — the algorithm runs until it gets there. And the merges are learned from the training corpus, which means a tokeniser trained on web text will behave oddly on medical or legal text. Hold that thought; it comes back in two slides.
- Let's run it by hand.

Training corpus:

```
["low", "lower", "lowest", "new", "newer", "newest"]
```

Initial character sequences (</w> marks end of word):

```
l o w</w>
```

```
l o w e r</w>
```

```
l o w e s t</w>
```

```
n e w</w>
```

```
n e w e r</w>
```

```
n e w e s t</w>
```

- [~3 min]
- Six words in our toy corpus: low, lower, lowest, new, newer, newest.
- We start by splitting everything into characters. The angle-bracket w marks the end of a word — we need that, otherwise the model cannot tell a word-final "er" from an "er" in the middle of something.
- Look at the six rows and, before I click on, see if you can spot which pair is most frequent. Everything else follows from that.

Merge 1: "l o" → "lo"
(appears in every "low*" word)

Merge 2: "lo w" → "low"
low</w>, low e r</w>, low e s t</w>, ...



BPE Worked Example — Merges 1–2

- [~3 min]
- Merge one: "l" plus "o" becomes "lo". It appears in every word starting with "low," so it is the most frequent pair.
- Merge two: now "lo" plus "w" becomes "low". Notice what just happened — the output of merge one became an input to merge two. The merges compose, and that is how BPE builds up from characters to whole words without anyone specifying what a word is.
- After two merges our rows read: low, low-e-r, low-e-s-t.

Merge 3: "n e" → "ne"

Merge 4: "ne w" → "new"

new</w>, new e r</w>, new e s t</w>, ...



BPE Worked Example — Merges 3–4

- [~2 min]
- Same thing on the other half of the corpus. "n" plus "e" becomes "ne", then "ne" plus "w" becomes "new".
- Four merges in and we have discovered two word stems — "low" and "new" — purely by counting. Nobody told the algorithm about morphology.

Merge 5: "e r</w>" → "er</w>"
(appears in both "lower" and "newer")

Result: "newest" tokenises as:
["new", "est</w>"]

Both pieces are common subwords already in the vocabulary — no UNK token needed.

- [~5 min]
- Merge five: "e" plus "r" plus end-of-word becomes "er" as a single word-final token, because it appears in both "lower" and "newer". The algorithm has now discovered a suffix.
- And the result: "newest" tokenises as "new" plus "est". Both pieces are common subwords already in the vocabulary. No UNK token needed.
- That is the whole point of the exercise. A word the model has arguably never needed to memorise gets represented perfectly from two reusable pieces.
- [If someone points out that merge five gives "er" and the result shows "est" — good catch, say so. The slide compresses a step; an analogous merge over "e", "s", "t" plus end-of-word produces "est" the same way. The mechanism is identical.]

Try It Yourself: Decomposing "unhappiness"

- Assuming a well-trained BPE vocabulary, what tokens would you expect?
 - "un" + "happiness", or "un" + "happy" + "ness", or something else entirely
- There is no single right answer -- it depends on the training corpus
 - BPE finds reusable, frequent pieces -- not linguistically principled morphemes

- [~4 min]
- Your turn. Assume a well-trained BPE vocabulary. What tokens would you expect for "unhappiness"?
- [Take answers. You will get "un" plus "happiness", and "un" plus "happy" plus "ness".]
- Both are plausible and there is no single right answer — it depends entirely on the training corpus. If "happiness" appeared often enough as a whole word, it stays whole. If it did not, it splits.
- Here is the thing I actually want you to take away. BPE finds reusable, frequent pieces. It does not find linguistically principled morphemes. It is a compression algorithm that happens to land near morphology, because frequent substrings and meaningful substrings correlate — but they are not the same thing, and where they diverge is where your model gets confused.

WordPiece vs. BPE

- WordPiece (BERT)
 - Merge criterion maximises the likelihood of the training corpus, not raw pair frequency
 - Marks continuation subwords with "##": "unhappiness" → ["un", "##happy", "##ness"]
- BPE (GPT-2)
 - Operates on bytes, not characters — fully language-agnostic by construction
 - Always merges the most frequent adjacent pair; uses a space prefix instead of ##

- [~5 min]
- Two variants you will meet constantly, so let's be precise about the difference.
- WordPiece — this is BERT. The merge criterion maximises the likelihood of the training corpus rather than raw pair frequency. In plain terms: it does not merge the most common pair, it merges the pair that most improves the model's ability to predict the corpus. Slightly smarter, slightly more expensive. And it marks continuation subwords with a double hash, so "unhappiness" becomes "un", "hash-hash-happy", "hash-hash-ness". The hashes tell you this piece attaches to the left.
- BPE as used in GPT-2 — operates on bytes, not characters. That sounds like a technicality and it is not: bytes mean it is fully language-agnostic by construction. There is no character it cannot represent, in any script, ever. It always merges the most frequent adjacent pair, and it marks word boundaries with a space prefix instead of the double hash.
- So: different merge criterion, different boundary marker, different base unit. When you load a pre-trained model and the tokens look strange, this slide is why.

SentencePiece

- A library (not a new algorithm) implementing both BPE and a unigram language-model approach
 - Treats whitespace as a regular symbol (`_`), making it fully language-agnostic
 - No language-specific pre-tokenisation step needed
- Used in T5, mT5, XLNet

- [~3 min]
- SentencePiece is the one people most often misclassify. It is a library, not a new algorithm. It implements both BPE and a unigram language-model approach.
- The genuinely useful idea is that it treats whitespace as a regular symbol — that underscore character — which means it needs no language-specific pre-tokenisation step at all. Everything before SentencePiece assumed you could split on spaces first. For Chinese, Japanese and Thai, you cannot. SentencePiece just does not make the assumption.
- Used in T5, mT5 and XLNet. If you work with multilingual models, you will meet it.

Vocabulary Size Trade offs

- Too small (e.g. 100 tokens)
 - Most words split into many pieces; sequences become very long, even common words get split
- Too large (e.g. 200,000 tokens)
 - Rare words become whole tokens, but the embedding matrix bloats — and OOV re-emerges for words outside training entirely
- Typical range: 30,000–50,000
 - BERT: 30,522 — GPT-2: 50,257

- [~5 min]
- Vocabulary size is a real engineering trade-off with two failure modes at the ends.
- Too small — say a hundred tokens. Almost everything splits into many pieces, sequences become very long, and even common words get chopped up. You have basically reinvented character tokenisation.
- Too large — say two hundred thousand. Rare words become whole tokens, which sounds nice, but the embedding matrix bloats, and out-of-vocabulary re-emerges anyway for anything outside your training data. You have paid for memorisation and still not solved the problem.
- The typical range is thirty to fifty thousand, and the two reference points worth remembering: BERT uses 30,522, GPT-2 uses 50,257.
- If anyone wonders about the odd numbers — those are the target size plus a handful of special tokens, things like padding, classification and mask markers.

Why Tokenisation Choices Matter

- Token budget
 - Every token costs one position in the model's context window
- Multilingual fairness
 - Morphologically rich languages (Finnish, Turkish) tokenise into more, shorter pieces — consuming more of the budget for the same meaning
- Downstream task alignment
 - A tokeniser trained on web text may split domain-specific terms (medical, legal, financial) into unusual pieces
- Worked calculation:
 - $300 \text{ words} \times \sim 1.3 \text{ subwords/word} = \sim 390 \text{ tokens}$ -- fits BERT's 512-token window
 - $600 \text{ words} \times \sim 1.3 \text{ subwords/word} = \sim 780 \text{ tokens}$ -- must be truncated (ECA A2 decision)

- [~7 min]
- Three reasons this matters practically rather than academically.
- Token budget. Every token costs one position in the model's context window. Tokenisation is not free — it directly determines how much of your document fits.
- Multilingual fairness. Morphologically rich languages like Finnish and Turkish tokenise into more and shorter pieces. Same meaning, more tokens, more of the budget consumed — and if you are paying per token, a literal higher price for the same content. That is a real equity issue and it is worth naming.
- Downstream task alignment. A tokeniser trained on general web text will split domain-specific terms — medical, legal, financial — into unusual pieces. Your model then has to reassemble the concept from fragments it has no reason to associate.
- Now the calculation on the slide, because you will need exactly this in the ECA.
- Three hundred words times roughly 1.3 subwords per word is about 390 tokens. That fits comfortably inside BERT's 512-token window.
- Six hundred words times 1.3 is about 780 tokens. That does not fit. You must truncate, or chunk, or pick a different model — and in ECA A2 you have to justify which.
- Memorise the 1.3 multiplier for English. It is a good enough back-of-envelope number to make architecture decisions with, and it is the kind of estimate I expect you to produce without opening a notebook.

Common Pitfalls

- Common pitfalls
 - Confusing vocabulary size with document count -- you choose a target BPE vocab size; BPE runs until it reaches it
 - Training BPE on dev/test data -- the tokeniser must be built from training data only
 - Reporting OOV as type count instead of token count -- token-level OOV rate is the primary metric
- Where this shows up in your assessments
 - TMA Q1b: implement and compare whitespace, regex, and BPE tokenisers on AG News
 - ECA A2: justify your tokenisation choice using OOV rate, vocabulary size, and sequence-length trade-offs

- [~5 min]
- Three pitfalls, all of which I will see if I do not say them now.
- Confusing vocabulary size with document count. You choose a target BPE vocabulary size and the algorithm runs until it reaches it. It has nothing to do with how many documents you have.
- Training BPE on dev or test data. The tokeniser must be built from training data only. This is a leakage bug, it is silent, and it will make your numbers look better than they are. Fit on train, apply to dev and test. Every time.
- Reporting OOV as a type count instead of a token count. Token-level OOV rate is the primary metric. Type-level makes rare words look like a catastrophe; token-level tells you what fraction of the actual text you are losing.
- And where it shows up: TMA question 1b, implement and compare whitespace, regex and BPE tokenisers on AG News. ECA A2, justify your tokenisation choice using OOV rate, vocabulary size and sequence-length trade-offs. Both of those are today's material, assessed directly.

The End

- [~4 min]
- Let me tie the two halves together, because they probably felt unrelated.
- First half: implementation is cheap now, judgment is expensive, and the value is upstream — in choosing problems and knowing what "solved" means.
- Second half: ambiguity, the pipeline, tokenisation, BPE. That is judgment in its concrete form. Knowing that a 600-word document will not fit in BERT's window before you write a line of code — that is the skill. Any model can produce a number. Knowing which number matters is the job.
- Before you go, four things.
- One: set up your GitHub repository before next session. Commit history is assessed from week one.
- Two: work the Kaggle tokenisation notebook from the tokenisation slide.
- Three: start TMA question 1a — the AG News data audit. Everything you need for it was in today's pipeline section.
- Four: read the study guide chapters one and two.
- Next week we fix the thing tokenisation cannot fix: that "good" and "great" are currently as unrelated as "good" and "refrigerator." Word embeddings.

- See you Thursday.